Понятие наследования.
Применение модификаторов доступа.
Вызов конструктора базового класса.
Приведение к базовому типу



IThub

Наследование

- позволяет создавать иерархические классификации
- унаследованный класс называется суперклассом, а класс, выполняющий наследование – подклассом
- **ключевое слово extends**
- в Java класс может наследоваться только от одного родительского класса
- позволяет использовать уже написанный код, что уменьшает дублирование
- все классы в Java являются подклассами Object

IThub

```java
public class Animal {

    private String color;
    private int age;
    private int weight;

    public Animal(String color, int age, int weight) {
        this.color = color;
        this.age = age;
        this.weight = weight;
    }

     // ... getters and setters …

    public void eat() {
        System.out.println("This animal eats food.");
    }
}
```

ключевое слово extends

```java
public class Dog extends Animal {

    private String name;
    private String breed;

    public Dog(String color, int age, int weight, String name,
               String breed) {
        super(color, age, weight);
        this.name = name;
        this.breed = breed;
    }

    // ... getters and setters …

    public void bark() {
        System.out.println("The dog barks");
    }
}
```

вызов конструктора базового класса идет в самом начале в конструкторе производного класса

IThub

Переопределение методов

- когда метод в подклассе имеет то же имя и сигнатуру типа, что и метод в его суперклассе - метод в подклассе переопределяет метод в суперклассе
- при вызове переопределенного метода через его подкласс всегда будет вызываться версия метода, определенная в подклассе
- переопределенные методы позволяют Java поддерживать полиморфизм во время выполнения
- ключевое слово final предотвращает переопределение методов

```java
public class Animal {

    private String color;
    private int age;
    private int weight;

    public Animal(String color, int age, int weight) {
        this.color = color;
        this.age = age;
        this.weight = weight;
    }

     // ... getters and setters …

    public void eat() {
        System.out.println("This animal eats food.");
    }
}
```

```java
public class Dog extends Animal {

    private String name;
    private String breed;

    public Dog(String color, int age, int weight, String name, String breed) {
        super(color, age, weight); // вызвать конструктор суперкласса
        this.name = name;
        this.breed = breed;
    }

    //... getters and setters …

    public void bark() {
        System.out.println("The dog barks");
    }

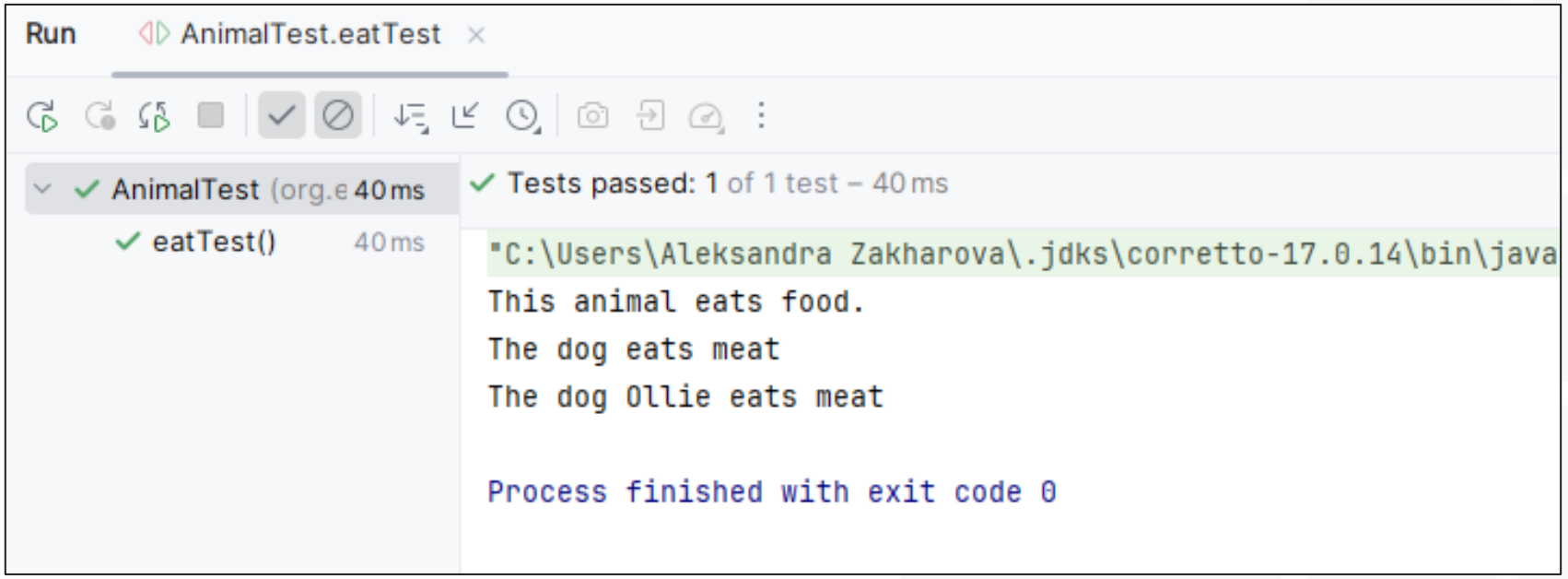
    @Override
    public void eat() {
        if (this.name != null && !this.name.isEmpty()) {
            System.out.println("The dog " + this.name + " eats meat");
        } else {
            System.out.println("The dog eats meat");
        }
    }
}
```

```java
@Test
public void eatTest() {
    Animal animal = new Animal();
    animal.eat();

    Dog dog = new Dog();
    dog.eat();

    Animal someDog = new Dog("gray", 5, 15, "Ollie", "Collie");
    someDog.eat();
}
```

Приведение к базовому типу (**Upcasting**) — это когда объект класса-наследника рассматривается как объект его родительского класса.



ОПЕРАТОР instanceof

- иногда полезно знать тип объекта во время выполнения, когда данные приходят извне
- оператор **instanceof** позволяет проверить, принадлежит ли объект указанному классу, с учётом наследования



```java
public abstract class A {

    abstract void printMe();

    public void printMeThere() {
        System.out.println("there is implemented method");
    }
}
```

```java
public class B extends A {

    @Override
    void printMe() {
        System.out.println("There is printMe in B class");
    }
}
```

```java
public class C extends A {

    @Override
    void printMe() {
        System.out.println("There is printMe in C class");
    }
}
```

```java
public class D extends B {

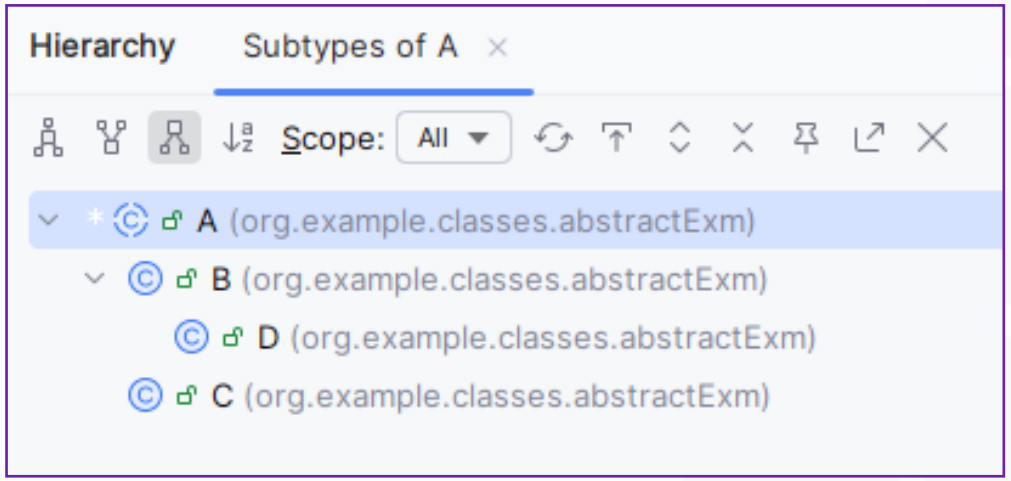
    public int dVar;
}
```

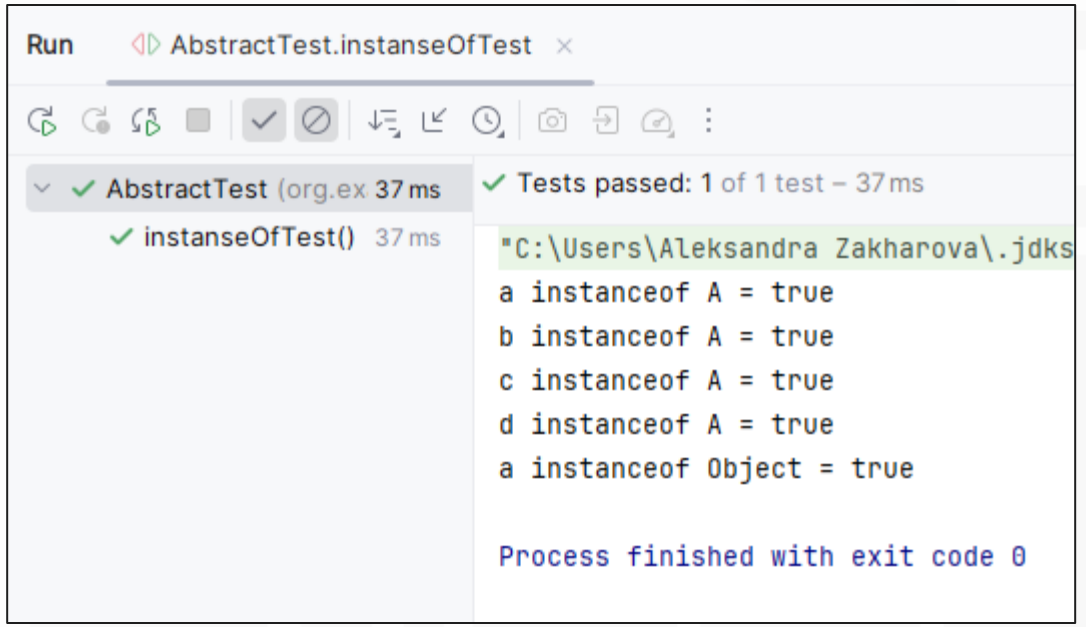
```java
@Test
public void instanseOfTest() {
    A a = new A() {
        @Override
        void printMe() {
            System.out.println("There is printMe in A class");
        }
    };

    B b = new B();
    C c = new C();
    D d = new D();

    boolean instanceOfResult = a instanceof A;
    System.out.println("a instanceof A = " + instanceOfResult);
    instanceOfResult = b instanceof A;
    System.out.println("b instanceof A = " + instanceOfResult);
    instanceOfResult = c instanceof A;
    System.out.println("c instanceof A = " + instanceOfResult);
    instanceOfResult = d instanceof A;
    System.out.println("d instanceof A = " + instanceOfResult);

    instanceOfResult = a instanceof Object;
    System.out.println("a instanceof Object = " + instanceOfResult);
}
```





Задание

IThub

1. Создайте базовый класс Employee. Поля:
- name — имя сотрудника (protected);
- salary — зарплата (private).

Добавьте:
- конструктор, который принимает имя и зарплату;
- метод showInfo(), который выводит информацию: Имя: ...Зарплата: ...

2. Создайте класс Developer, который наследуется от Employee
Добавьте поле: language — основной язык программирования.
В конструкторе: вызовите конструктор базового класса через super().
Переопределите метод showInfo(): Имя: Алекс Зарплата: 150000 Язык: Java

3. Создайте класс Manager, который тоже наследуется от Employee
Добавьте поле: teamSize — количество сотрудников в команде.
Переопределите метод showInfo(): Имя: Ольга Зарплата: 180000 Команда: 10 человек

4. В классе Main выполните:
Создайте разработчика:
Developer dev = new Developer("Алекс", 150000, "Java")
Создайте менеджера:
Manager manager = new Manager("Ольга", 180000, 10)
Приведите объекты к базовому типу (Employee employee1 = dev; Employee employee2 = manager)
Вызовите: employee1.showInfo(); employee2.showInfo();

Дополнительные вопросы

- Почему переменная типа Employee может хранить объект Developer?
- Почему поле salary нельзя использовать напрямую в классе Developer?
- Какой метод будет вызван:

```
Employee employee = new Developer("Иван", 120000, "Python");
employee.showInfo();
```

Метод Employee или Developer? Почему?